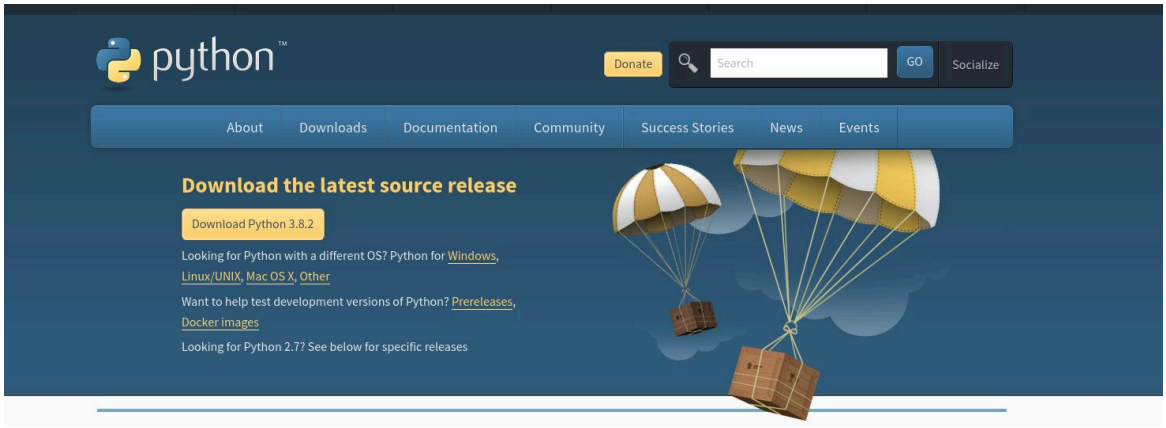
Introduction

This is the UBTECH AlphaMini robot python sdk.

I. The Installation Environment

1.1 Download and install python

- Download from: https://www.python.org/downloads/windows/



- Windows OS, “Configure environment variables” is checked during installation in default.
- Linux/Unix OS, refer to https://wiki.python.org/moin/BeginnersGuide/Download for download and installation instructions.
- Verify installation
- python –version & pip –version

```
[logic@Logic mini-python-sdk使用文档]$ python --version & pip --version
[1] 644178
Python 3.8.2
pip 19.3 from /usr/lib/python3.8/site-packages/pip (python 3.8)
[1]+  Done                    python --version
```

1.2 Install/uninstall AlphaMini sdk

The AlphaMini sdk can be uninstalled and installed by the following two commands.

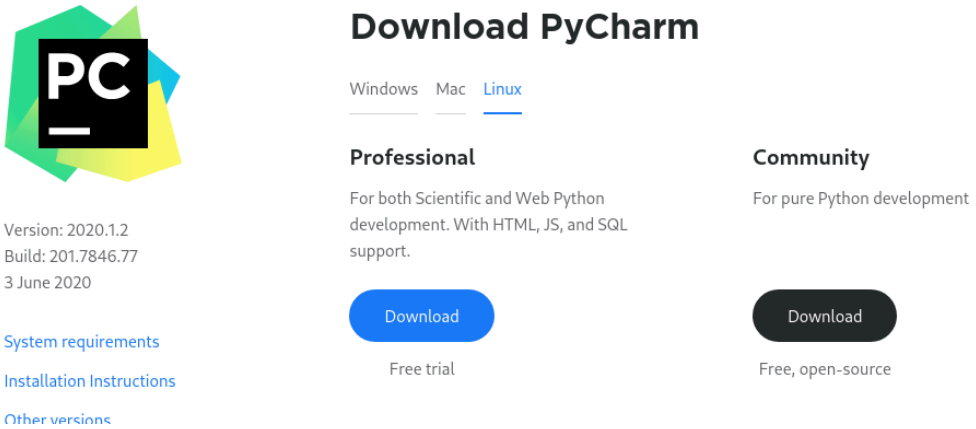
- pip install alphamini
- pip uninstall alphamini

You can use `pip show alphamini` to view the installed sdk version, Please confirm whether to update to the latest SDK?

After installing the SDK, if the robot cannot be searched using the AlphaMini sdk API, please refer to the link : http://docs.ubtrobot.com/alphamini/python-sdk-en/qa.html

1.3 Install PyCharm

- Download from: https://www.jetbrains.com/pycharm/download/



- Or you can choose install professional PyCharm, make sure you can get an activation code.

1.4 Prepare one AlphaMini.

- Make sure the version of AlphaMini is not lower than V1.4.0.
- Make sure the robot and PC are on the same LAN.

1.5 Download the demo and run it.

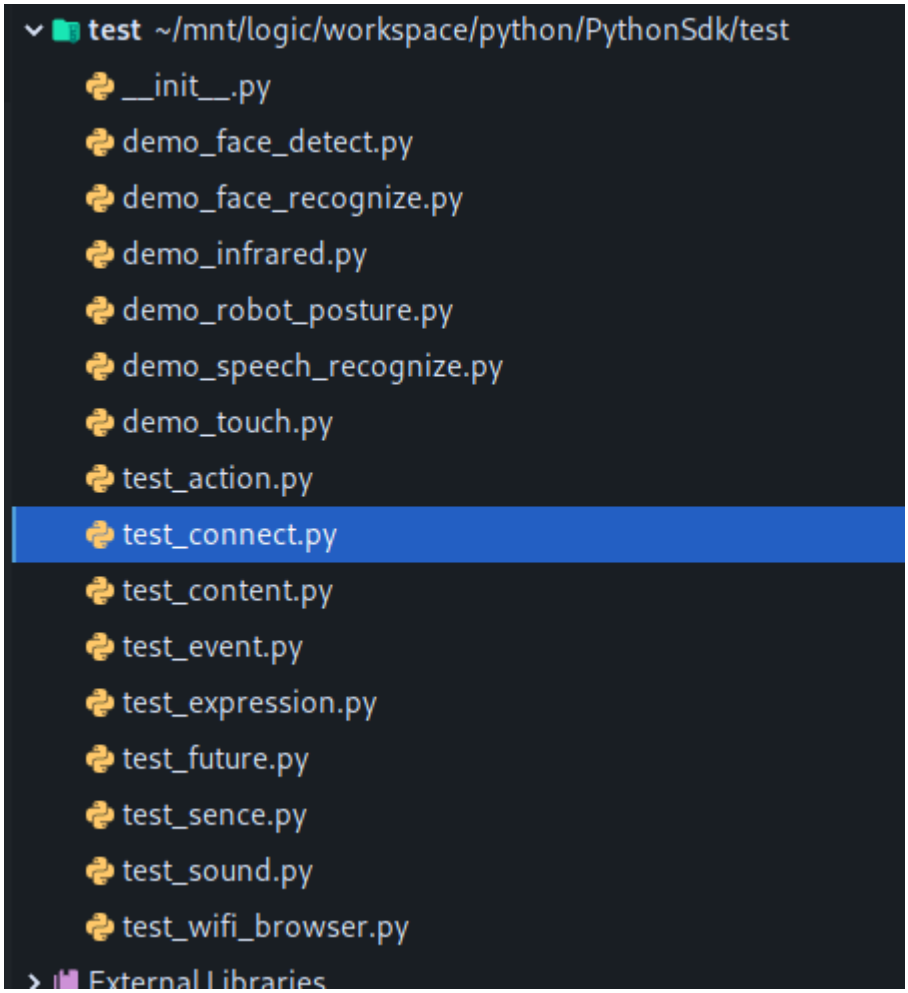
- Download from: https://github.com/marklogg/mini_demo.git

```
git clone git@github.com:marklogg/mini_demo.git
```

- Please use the `git` command to switch the project to the `oversea_develop` branch, which contains English comments

```
git checkout oversea_develop
```

- Import the demo into pycharm with the following structure.



- Select the file and right-click to run “Run xxx.py”

II. Introduction to the API

2.1 Public Interface

- Importing Packages :

```
import mini.mini_sdk as MiniSdk
```

2.1.1. Scan for robots on a LAN

Before calling `MiniSdk.get_device_by_name()` to find robots on a LAN, the type of robot should be set by `MiniSdk.RobotType.EDU`.

If your robot cannot be searched, please refer to the link : http://docs.ubtrobot.com/alphamini/python-sdk-en/qa.html.

```
  # The default log level is Warning, set to INFO
  MiniSdk.set_log_level(logging.INFO)

  #Before calling MiniSdk.get_device_by_name, the type of robot should be set by command
MiniSdk.RobotType.EDU
  # Set robot type
  MiniSdk.set_robot_type(MiniSdk.RobotType.EDU)

  # Search for a robot with the robot's serial number (on the back of robot), the length of
the serial number is arbitrary, it is recommended more than 5 characters to match exactly, 10
seconds timeout.
  # Search results for WiFiDevice, including robot name, ip, port, etc.
  async def test_get_device_by_name(). result: WiFiDevice = await
MiniSdk.get_device_by_name("00018", 10):
    result: WiFiDevice = await MiniSdk.get_device_by_name("00018", 10)
    print(f "test_get_device_by_name result:{result}")
    return result

  # Search for the robot with the specified serial number (behind the robot's ass),
  async def test_get_device_list(). results = await MiniSdk.get_device_list(10):
    results = await MiniSdk.get_device_list(10)
    print(f "test_get_device_list results = {results}")
    return results
```

2.1.2. Connect the robot

`MiniSdk.connect()` is used to connect the robot based on the scannning results.

```
# MiniSdk.connect returns bool, ignoring the return value.
async def test_connect(dev: WiFiDevice):
    await MiniSdk.connect(dev)
```

2.1.3. Get the robot into programming mode

Get the robot into programming mode, which prevents the robot from being interrupted by other skills while executing commands.

```
# Enter programming mode, the robot has a tts broadcast, here through asyncio.sleep let the
current concatenation wait for 6 seconds to return, let the robot broadcast finished.
async def test_start_run_program(). await StartRunProgram().execute():
    await StartRunProgram().execute()
    await asyncio.sleep(6)
```

2.1.4. Disconnect and release resources

```
async def shutdown():
    await asyncio.sleep(1)
    await MiniSdk.release()
```

2.1.5. sdk log switch

```
# The default log level is Warning, set to INFO.
MiniSdk.set_log_level(logging.INFO)
```

2.2 Introduction to API

- importing api

```
    from mini.apis import *
```

- each `api` returns a tuple, the first element of the tuple is a `bool`, the second element is a `response(protobuf)`, if the first element is False, the second element is None.
- Each `api` has an `execute()` method, which is an `async` method, that triggers execution.
- Each api has an `is_serial` parameter, in default the value is True, which means the interface can be executed serially, and `await` will get the result. If `is_serial` =False, which means just send the command to the robot, `await` does not need to wait for the robot to execute the result and then return.

2.2.1 Sound Control

```python
# Testing text synthesized sounds
async def test_play_tts():
    """test_play_tts

    Make the robot start playing a tts saying "Hello, I'm AlphaMini, la-la-la-la" and wait for
the result.

    #ControlTTSResponse.isSuccess : if successful

    #ControlTTSResponse.resultCode : return code

    """
    # is_serial:serial execution
    # text: the text to combine
    block: StartPlayTTS = StartPlayTTS(text="Hello, I'm AlphaMini, la-la-la")
    # Returns a tuple, response is a ControlTTSResponse
    (resultType, response) = await block.execute()

    print(f'test_play_tts result: {response}')
    # StartPlayTTS block response contains resultCode and isSuccess
    # If resultCode ! =0 can be queried by errors.get_speech_error_str(response.resultCode))
    print('resultCode = {0}, error = {1}'.format(response.resultCode,
errors.get_speech_error_str(response.resultCode)))

# Test playback sound (online)
async def test_play_online_audio():
    """test_play_online_audio

    Make the robot play an online sound effect, e.g.:
"http://hao.haolingsheng.com/ring/000/995/52513bb6a4546b8822c89034afb8bacb.mp3"

    Supported formats are mp3, amr, wav, etc.

    And wait for the results.

    #PlayAudioResponse.isSuccess : whether it succeeds or not

    #PlayAudioResponse.resultCode : return code

    """
    # Play sound effects, url is the list of sound effects to play.
    block: PlayAudio = PlayAudio(
        url="http://hao.haolingsheng.com/ring/000/995/52513bb6a4546b8822c89034afb8bacb.mp3",
        storage_type=AudioStorageType.NET_PUBLIC)
    # response is a PlayAudioResponse
    (resultType, response) = await block.execute()

    print(f'test_play_online_audio result: {response}')
    print('resultCode = {0}, error = {1}'.format(response.resultCode,
errors.get_speech_error_str(response.resultCode)))

# Test the sound resources of the acquisition robot
async def test_get_audio_list():
    """Test to play local sound effects

    Make the bot play a locally built-in sound effect named "read_016" and wait for the
result.

    #PlayAudioResponse.isSuccess : whether it succeeds or not

    #PlayAudioResponse.resultCode : return code
```

```
    """

    block: PlayAudio = PlayAudio(
        url="read_016",
        storage_type=AudioStorageType.PRESET_LOCAL)
    # response is a PlayAudioResponse
    (resultType, response) = await block.execute()

    print(f'test_play_local_audio result: {response}')
    print('resultCode = {0}, error = {1}'.format(response.resultCode,
errors.get_speech_error_str(response.resultCode)))

# Test to stop the sound that's playing
async def test_stop_audio():
    """The test stops all audio that is playing

    First play a tts, after 3s, stop all sound effects and wait for the result

    #StopAudioResponse.isSuccess : whether or not it succeeds

    #StopAudioResponse.resultCode : return code

    """
    # Set is_serial=False, which means you just send the command to the robot, and the wait
doesn't need to wait for the robot to return the result.
    block: StartPlayTTS = StartPlayTTS(is_serial=False, text="You let me talk, let me talk,
don't interrupt me, don't interrupt me, don't interrupt me.")
    response = await block.execute()
    print(f'test_stop_audio.play_tts: {response}')
    await asyncio.sleep(3)

    # Stop all sound
    block: StopAllAudio = StopAllAudio()
    (resultType, response) = await block.execute()

    print(f'test_stop_audio:{response}')

    block: StartPlayTTS = StartPlayTTS(text="For the second time, you let me talk, let me
talk, don't interrupt me, don't interrupt me, don't interrupt me.")
    asyncio.create_task(block.execute()))
    print(f'test_stop_audio.play_tts: {response}')
    await asyncio.sleep(3)
```

2.2.2 Infrared/vision/camera etc. sensor interfaces

- Obtain primary sensor results through these sensor interfaces.

```python
# Testing for face detection
async def test_face_detect():
    """test face count detection
    Detect the number of faces, 10s timeout, and wait for a response.

    #FaceDetectResponse.count : number of faces

    #FaceDetectResponse.isSuccess : Success or Not

    #FaceDetectResponse.resultCode : Return Code

    """
    # timeout: Specify the duration of the detection
    block: FaceDetect = FaceDetect(timeout=10)
    # response: FaceDetectResponse
    (resultType, response) = await block.execute()

    print(f'test_face_detect result: {response}')


# Test object recognition: flower recognition, 10s timeout
async def test_object_recognise_flower():
    """Test object (flower) recognition

    Have the robot identify the flower (you have to manually put the flower or a photo of the
flower in front of the robot), time out for 10s, and wait for the results

    #RecogniseObjectResponse.objects : array of object names [str]

    #RecogniseObjectResponse.isSuccess : whether or not it succeeds

    #RecogniseObjectResponse.resultCode : Return Code

    """
    # object_type: Supports FLOWER, FRUIT, GESTURE
    block: ObjectRecognise = ObjectRecognise(object_type=ObjectRecogniseType.FLOWER,
timeout=10)
    # response : RecogniseObjectResponse
    (resultType, response) = await block.execute()

    print(f'test_object_recognise_flower result: {response}')


# Test object recognition: identify fruit, 10s timeout
async def test_object_recognise_fruit():
    """Test object (fruit) recognition

    Have the robot identify the flower (you have to manually put the fruit or a picture of the
fruit in front of the robot), time out for 10s, and wait for the result

    #RecogniseObjectResponse.objects : array of object names [str]

    #RecogniseObjectResponse.isSuccess : whether or not it succeeds

    #RecogniseObjectResponse.resultCode : Return Code

    """
    # object_type: Supports FLOWER, FRUIT, GESTURE objects.
    block: ObjectRecognise = ObjectRecognise(object_type=ObjectRecogniseType.FRUIT,
timeout=10)
    # response : RecogniseObjectResponse
    (resultType, response) = await block.execute()
```

```
    print(f'test_object_recognise_fruit result: {response}')


# Test object recognition: gesture recognition, 10s timeout
async def test_object_recognise_gesture():
    ""test object (gesture) recognition

    Get the robot to recognize the flower (you have to manually gesture in front of the
robot), time out for 10s, and wait for the result.

    #RecogniseObjectResponse.objects : array of object names [str]

    #RecogniseObjectResponse.isSuccess : whether or not it succeeds

    #RecogniseObjectResponse.resultCode : Return Code

    """
    # object_type: Supports FLOWER, FRUIT, GESTURE
    block: ObjectRecognise = ObjectRecognise(object_type=ObjectRecogniseType.GESTURE,
timeout=10)
    # response : RecogniseObjectResponse
    (resultType, response) = await block.execute()

    print(f'test_object_recognise_gesture result: {response}')


 Test photography
async def test_take_picture():
    """test_take_picture()

    Have the robot take pictures immediately and wait for the results

    #TakePictureResponse.isSuccess : Success or Not

    #TakePictureResponse.code : return code

    #TakePictureResponse.picPath : The path where the photo is stored in the bot.

    """
    # response: TakePictureResponse
    # take_picture_type: IMMEDIATELY-immediately, FINDFACE-find the face and take the picture.
    (resultType, response) = await
TakePicture(take_picture_type=TakePictureType.IMMEDIATELY).execute()

    print(f'test_take_picture result: {response}')


# Testing facial recognition
async def test_face_recognise():
    """test face recognition

    Have the robot perform face recognition inspection, time out 10s, and wait for the results


    #FaceRecogniseResponse.faceInfos : [FaceInfoResponse] face information array

        FaceInfoResponse.id : face id

        FaceInfoResponse.name : name, default name is "stranger" if it's a stranger

        FaceInfoResponse.gender : Gender

        FaceInfoResponse.age : Age
```

```
    #FaceRecogniseResponse.isSuccess : whether or not successful

    #FaceRecogniseResponse.resultCode : Return Code

    Returns:

    """
    # response : FaceRecogniseResponse
    (resultType, response) = await FaceRecognise(timeout=10).execute()

    print(f'test_face_recognise result: {response}')


# Test to acquire infrared detection distance
async def test_get_infrared_distance():
    """test_infrared_distance_detection

    Get the IR distance detected by the current robot and wait for the result.

    #GetInfraredDistanceResponse.distance : Infrared distance

    """
    # response: GetInfraredDistanceResponse
    (resultType, response) = await GetInfraredDistance().execute()

    print(f'test_get_infrared_distance result: {response}')


# Test to get the number of faces currently registered in the bot
async def test_get_register_faces():
    """Test gets registered faces information

    Get all the faces registered in the bot and wait for the results.

    #GetRegisterFacesResponse.faceInfos : [FaceInfoResponse] face information array

        #FaceInfoResponse.id : face id

        #FaceInfoResponse.name : Name

        #FaceInfoResponse.gender : gender

        #FaceInfoResponse.age : age

    #GetRegisterFacesResponse.isSuccess : whether it succeeds or not

    #GetRegisterFacesResponse.resultCode : Return Code

    Returns:

    """
    # reponse : GetRegisterFacesResponse
    (resultType, response) = await GetRegisterFaces().execute()

    print(f'test_get_register_faces result: {response}')
```

2.2.3 Robot Expressiveness

```python
# Test the eyes to make a face
async def test_play_expression():
    """Test play expressions

    Let the bot play a built-in emoticon called "codemao1" and wait for the reply results!

    #PlayExpressionResponse.isSuccess : Success or not

    #PlayExpressionResponse.resultCode : Return Code

    """
    block: PlayExpression = PlayExpression(address_name="codemao1")
    # response: PlayExpressionResponse
    (resultType, response) = await block.execute()

    print(f'test_play_expression result: {response}')


# Test, make the robot dance/stop dancing
async def test_control_behavior():
    """test control expressivity

    Ask the robot to start a dance called "dance_0004" and wait for the response.

    """
    # control_type: START, STOP
    block: StartBehavior = StartBehavior(name="dance_0004en")
    # response ControlBehaviorResponse
    (resultType, response) = await block.execute()

    print(f'test_control_behavior result: {response}')
    print(
        'resultCode = {0}, error = {1}'.format(response.resultCode,
errors.get_express_error_str(response.resultCode)))


# Test, set the mouth light to green Always on
async def test_set_mouth_lamp():
    # mode: mouthlamp mode, 0: normal mode, 1: breath mode

    # color: mouth light color, 1: red, 2: green

    # duration: duration in milliseconds, -1 means always on.

    # breath_duration: duration of a blink in milliseconds

    """Test Set Mouth Light

    Set the robot's mouth light to normal mode, green and always on for 3s, and wait for the
reply result.

    When mode=NORMAL, the duration parameter works, indicating how long it will be always on.

    When mode=BREATH, the breath_duration parameter works, indicating how often to breathe

    #SetMouthLampResponse.isSuccess : Success or Not

    #SetMouthLampResponse.resultCode : return code

    """

    block: SetMouthLamp = SetMouthLamp(color=MouthLampColor.GREEN, mode=MouthLampMode.NORMAL,
                                       duration=3000, breath_duration=1000)
```

```
    # response:SetMouthLampResponse
    (resultType, response) = await block.execute()

    print(f'test_set_mouth_lamp result: {response}')


# Test, switch mouth light
async def test_control_mouth_lamp():
    """test_control_mouth_lamp

    Have the robot turn off its mouth light and wait for the results.

    #ControlMouthResponse.isSuccess : whether it succeeds or not

    #ControlMouthResponse.resultCode : return code

    """
    # is_open: True,False
    # response :ControlMouthResponse
    (resultType, response) = await ControlMouthLamp(is_open=False).execute()

    print(f'test_control_mouth_lamp result: {response}')
```

2.2.4 Third-party content interfaces

- Translate

```
# Test the translation interface
async def test_start_translate():
    """Translate demo

    Use Baidu translation to translate "Jacky Cheung" from Chinese to English, and wait for
the result to be announced by the robot.

    #TranslateResponse.isSuccess : Success or not

    #TranslateResponse.resultCode : Return Code

    # query: keyword

    # from_lan: source language

    # to_lan: target language

    # platform: GOOGLE

    """

    block: StartTranslate = StartTranslate(query="Jacky Cheung", from_lan=LanType.CN,
to_lan=LanType.EN)
    # response: TranslateResponse
    (resultType, response) = await block.execute()

    print(f'test_start_translate result: {response}')
```

2.2.5 Motion Control

```
# Test, execute an action file
async def test_play_action():
    """Execute an action demo

    Control the robot to perform a named local (built-in/custom) action and wait for a
response from the result of the action

    The name of the action can be obtained from the GetActionList.

    #PlayActionResponse.isSuccess : whether it succeeds or not

    #PlayActionResponse.resultCode : Return Code

    """
    # action_name: Action file name, get action supported by robot via GetActionList.
    block: PlayAction = PlayAction(action_name='018')
    # response: PlayActionResponse
    (resultType, response) = await block.execute()

    print(f'test_play_action result:{response}')


# Test, control robot, move forward/backward/left/right
async def test_move_robot():
    """Control robot move demo

    Control the robot to move 10 steps to the left (LEFTWARD) and wait for the result.

    #MoveRobotResponse.isSuccess : whether it succeeds or not

    #MoveRobotResponse.code : return code

    """
    # step: move a few steps
    # direction: direction, enumeration type
    block: MoveRobot = MoveRobot(step=10, direction=MoveRobotDirection.LEFTWARD)
    # response : MoveRobotResponse
    (resultType, response) = await block.execute()

    print(f'test_move_robot result:{response}')


# Test, get a list of supported action files
async def test_get_action_list():
    """Get action list demo

    Get a list of actions built into the bot and wait for reply results.

    """
    # action_type: INNER is a non-modifiable action file built into the robot, CUSTOM is an
action file placed in sdcard/customize/action that can be modified by the developer.
    block: GetActionList = GetActionList(action_type=RobotActionType.INNER)
    # response:GetActionListResponse
    (resultType, response) = await block.execute()

    print(f'test_get_action_list result:{response}')
```

2.2.6 Event Listening Class Interface

2.2.6.1 Touch Listening

```
# Test, touch listen.
async def test_ObserveHeadRacket():
    # Create a listen
    observer: ObserveHeadRacket = ObserveHeadRacket()

    # Event processors
    # ObserveHeadRacketResponse.type:
    # @enum.unique
    # class HeadRacketType(enum.Enum):
    # SINGLE_CLICK = 1 # Click on the
    # LONG_PRESS = 2 # Press and hold...
    # DOUBLE_CLICK = 3 # DOUBLE_CLICK
    def handler(msg: ObserveHeadRacketResponse):
        # Stop listening when an event is monitored,
        observer.stop()
        print("{0}".format(str(msg.type)))
        # Execute a dance
        asyncio.create_task(__dance()))

observer.set_handler(handler)
    #begin
    observer.start()
    await asyncio.sleep(0)


async def __dance():
    await ControlBehavior(name="dance_0002").execute()
    # end event_loop
    asyncio.get_running_loop().run_in_executor(None, asyncio.get_running_loop().stop)

# Program entry
if __name__ == '__main__':
    device: WiFiDevice =
asyncio.get_event_loop().run_until_complete(test_get_device_by_name())
    if device:
        asyncio.get_event_loop().run_until_complete(test_connect(device))
        asyncio.get_event_loop().run_until_complete(test_start_run_program())
        asyncio.get_event_loop().run_until_complete(test_ObserveHeadRacket())
        asyncio.get_event_loop().run_forever() # If you define an event listener, you must
make event_loop.run_forver.
        asyncio.get_event_loop().run_until_complete(shutdown())
```

2.2.6.2 Voice Recognition Listening

```
async def __tts():
    block: PlayTTS = PlayTTS(text="Hello, I'm AlphaMini. La-la-la-la-la...")
    response = await block.execute()
    print(f'tes_play_tts: {response}')

# Test listening for voice recognition
async def test_speech_recognise():
    # Listening object for voice
    observe: ObserveSpeechRecognise = ObserveSpeechRecognise()

    # processors
    # SpeechRecogniseResponse.text
    # SpeechRecogniseResponse.isSuccess
    # SpeechRecogniseResponse.resultCode
    def handler(msg: SpeechRecogniseResponse):
        print(f'=======handle speech recognise:{msg}')
        print("{0}".format(str(msg.text)))
        if str(msg.text) == "AlphaMini":
            #I hear "AlphaMini", tts says hello.
            asyncio.create_task(__tts())

        elif str(msg.text) == "end":
            #Listen to the end, stop listening.
            observe.stop()
            # End event_loop
            asyncio.get_running_loop().run_in_executor(None, asyncio.get_running_loop().stop)

    observe.set_handler(handler)
    #begin
    observe.start()
    await asyncio.sleep(0)

if __name__ == '__main__':
    device: WiFiDevice =
asyncio.get_event_loop().run_until_complete(test_get_device_by_name())
    if device:
        asyncio.get_event_loop().run_until_complete(test_connect(device))
        asyncio.get_event_loop().run_until_complete(test_start_run_program())
        asyncio.get_event_loop().run_until_complete(test_speech_recognise())
        asyncio.get_event_loop().run_forever() # If you define an event listener, you must
make event_loop.run_forver.
        asyncio.get_event_loop().run_until_complete(shutdown())
```

2.2.6.3 infrared monitoring

```
async def test_ObserveInfraredDistance():
    #Infrared monitoring of objects
    observer: ObserveInfraredDistance = ObserveInfraredDistance()

    # Defining the Processor
    # ObserveInfraredDistanceResponse.distance
    def handler(msg: ObserveInfraredDistanceResponse):
        print("distance = {0}".format(str(msg.distance)))
        if msg.distance < 500:
            observer.stop()
            asyncio.create_task(__tts())

    observer.set_handler(handler)
    observer.start()
    await asyncio.sleep(0)

async def __tts():
    result = await PlayTTS(text="Is someone there, who are you").execute()
    print(f"tts over {result}")
    asyncio.get_running_loop().run_in_executor(None, asyncio.get_running_loop().stop)


if __name__ =='__main__':
    device: WiFiDevice =
asyncio.get_event_loop().run_until_complete(test_get_device_by_name())
    if device:
        asyncio.get_event_loop().run_until_complete(test_connect(device))
        asyncio.get_event_loop().run_until_complete(test_start_run_program())
        asyncio.get_event_loop().run_until_complete(test_ObserveInfraredDistance())
        asyncio.get_event_loop().run_forever() # Defines the event listener object, must let
event_loop.run_forver
        asyncio.get_event_loop().run_until_complete(shutdown())
```

2.2.6.4 Face recognition/detection monitoring

```python
#Test, if the registered face is detected, the incident will be reported, if it is a stranger, it will return to "stranger"
async def test_ObserveFaceRecognise():
    observer: ObserveFaceRecognise = ObserveFaceRecognise()


    # FaceRecogniseTaskResponse.faceInfos: [FaceInfoResponse]
    # FaceInfoResponse.id, FaceInfoResponse.name,FaceInfoResponse.gender,FaceInfoResponse.age
    # FaceRecogniseTaskResponse.isSuccess
    # FaceRecogniseTaskResponse.resultCode
    def handler(msg: FaceRecogniseTaskResponse):
        print(f"{msg}")
        if msg.isSuccess and msg.faceInfos:
            observer.stop()
            asyncio.create_task(__tts(msg.faceInfos[0].name))

    observer.set_handler(handler)
    observer.start()
    await asyncio.sleep(0)


async def __tts(name):
    await PlayTTS(text=f'Hello, {name}').execute()
    asyncio.get_running_loop().run_in_executor(None, asyncio.get_running_loop().stop)

# Face detection, if a face is detected, the incident will be reported
async def test_ObserveFaceDetect():
    observer: ObserveFaceDetect = ObserveFaceDetect()

    # FaceDetectTaskResponse.count
    # FaceDetectTaskResponse.isSuccess
    # FaceDetectTaskResponse.resultCode
    def handler(msg: FaceDetectTaskResponse):
        print(f"{msg}")
        if msg.isSuccess and msg.count:
            observer.stop()
            asyncio.create_task(__tts(msg.count))

    observer.set_handler(handler)
    observer.start()
    await asyncio.sleep(0)


async def __tts(count):
    await PlayTTS(text=f'There seems to be {count} people in front of me').execute()
    asyncio.get_running_loop().run_in_executor(None, asyncio.get_running_loop().stop)
```

2.2.6.5 Posture detection and monitoring

```
# Test, attitude detection
async def test_ObserveRobotPosture():
    #Create listening object
    observer: ObserveRobotPosture = ObserveRobotPosture()
    # Event handler
    # ObserveFallClimbResponse.status
    # STAND = 1; //Stand
    # SPLITS_LEFT = 2; //Left lunge
    # SPLITS_RIGHT = 3; //Right lunge
    # SITDOWN = 4; //Sit down
    # SQUATDOWN = 5; //Squat down
    # KNEELING = 6; //Kneel down
    # LYING = 7; //Lying on your side
    # LYINGDOWN = 8; //Lying down
    # SPLITS_LEFT_1 = 9; //Left split
    # SPLITS_RIGHT_2 = 10;//right split
    # BEND = 11;//Bent over
    def handler(msg: ObserveFallClimbResponse):
        print("{0}".format(msg))
        if msg.status == 8 or msg.status == 7:
            observer.stop()
            asyncio.create_task(__tts())

    observer.set_handler(handler)
    #start
    observer.start()
    await asyncio.sleep(0)


async def __tts():
    await PlayTTS(text="I fell down").execute()
    asyncio.get_running_loop().run_in_executor(None, asyncio.get_running_loop().stop)


if __name__ =='__main__':
    device: WiFiDevice =
asyncio.get_event_loop().run_until_complete(test_get_device_by_name())
    if device:
        asyncio.get_event_loop().run_until_complete(test_connect(device))
        asyncio.get_event_loop().run_until_complete(test_start_run_program())
        asyncio.get_event_loop().run_until_complete(test_ObserveRobotPosture())
        asyncio.get_event_loop().run_forever() # Defines the event listener object, must let
event_loop.run_forver
        asyncio.get_event_loop().run_until_complete(shutdown())
```

III. Command Line Tools

Because the robot has a built-in python execution environment (the robot supports python3.8), the program developed by the developer can be installed in the robot through the `pip` command (Python package management tool).

Developers can use the command line tool of the SDK to install the program into the robot and trigger execution after debugging their python program on the PC. What the command line tool can do:

- Using command line tools, users can package the compiled `py` program into a `pypi` compressed package and send it to the robot for installation.
- Using command line tools, users can `add, delete, modify, and check` the `pypi` installation package in the robot.
- Using the command line tool, the user also triggers the `py` program to execute on the robot side.

In addition, these tools have corresponding python interface, which can be imported through `import mini.pkg_tool as Tool`.

The AlphaMini sdk version that supports the command line tool should not lower than V1.0.0, AlphaMini robot version should be v1.4.0 or higher.

The developer packages the project into a `.whl` or `.egg` compressed file, and needs to write the `setup.py` configuration file in the root directory of the project. For how to configure `setup.py`, please refer to the configuration of the `setup.py` file in the demo.

3.1 Packaging

3.1.1 Configuration environment

First of all, make sure that the latest `setuptools`, `wheel` and `twine` are installed on the PC. The installation/update commands were shown as below.

```
python -m pip install --user --upgrade setuptools wheel twine
```

3.1.2 setup.py

Write `setup.py` in your project directory, refer to demo:

```
import setuptools
setuptools.setup(
    name="tts_demo", # Your library (program) name
    version="0.0.2", # version number
    author='Gino Deng', # developer name,
    author_email='jingjing.deng@ubtrobot.com', # Email, optional
    description="demo with mini_sdk", #short description
    long_description='demo with mini_sdk,xxxxxxx', #long description, optional
    long_description_content_type="text/markdown", #The format of the long description
content, optional
    license="GPLv3", # copyright
    packages=setuptools.find_packages(), #*Note*: By default, all subdirectories containing
__init__.py in the project directory are taken here
    classifiers=[
        "Programming Language :: Python :: 3",
        "Programming Language :: Python :: 3.6",
        "Programming Language :: Python :: 3.7",
        "Programming Language :: Python :: 3.8",
        "License :: OSI Approved :: MIT License",
        "Operating System :: OS Independent",
    ],
    install_requires=[ # Dependent module
        'alphamini',
    ],
    entry_points={
        'console_scripts': [# Command line entry definition
            'tts_demo = play_tts.test_playTTS:main'
        ],
    },
)
```

The parameters of setup.py are as follows:

```
--name Package name
--version (-V) package version
--author Author of the program
--author_email The email address of the program's author.
--maintainer maintainer
--maintainer_email The email address of the maintainer.
--url The program's official website address
--Authorization information for the license program
--description A brief description of the program.
--long_description Detailed description of the program
List of software platforms for the --platforms program
--List of classifiers to which the program belongs.
--keywords List of keywords for the program
--packages Directory of packages to be processed (folder containing __init__.py)
--py_modules List of python files to be packaged.
--download_url The download address of the program.
--cmdclass
--data_files The data files to be packaged, such as images, configuration files, etc.
--scripts List of footsteps to be performed during installation
--package_dir tells setuptools which files in which directories are mapped to which source
packages. An example: package_dir = {'': 'lib'}, indicating that the modules in the "root
package" are in the lib directory.
--define which modules are dependent
--provides definitions for which modules can be provided with dependencies
---find_packages() It's easy to manually add packages to a simple project, but we just used
this function to search for packages containing __init__.py in the same directory as setup.py
by default.

                        In fact, you can put the packages in a single src directory, and
there may be aaa.txt files and data folders inside the package. In addition, you can exclude
specific packages from the

                        find_packages(exclude=["*.tests", "*.tests.*", "tests.*", "tests"])

--install_requires = ["requests"] Dependencies to install
--entry_points Dynamic discovery of services and plugins, discussed in detail below
```

In the following entry_points: console_scripts specifies the name of the command line tool; in `tts_demo = play_tts.test_playTTS:main`, the name of the toolkit is specified before the equals sign, and the entry address of the program is specified after the equals sign.

```
entry_points={
        'console_scripts': [ # Command-line entry definition
            'tts_demo = play_tts.test_playTTS:main'
        ],
    }
```

There can be multiple records here so that a single project can make multiple command line tools, such as.

```
#... Omit others
entry_points = {
         'console_scripts': [
             'foo = demo:test',
             'bar = demo:test',
         ]}
```

3.1.3 Packing

After writing `setup.py` in your project directory, you can package it with a command line utility

- Mode 1: Use python command to call setup.py to package, run the following shell command in the project directory.

```
        python setup.py sdist bdist_wheel
```

- Mode 2: Through the command line tool in AlphaMini.

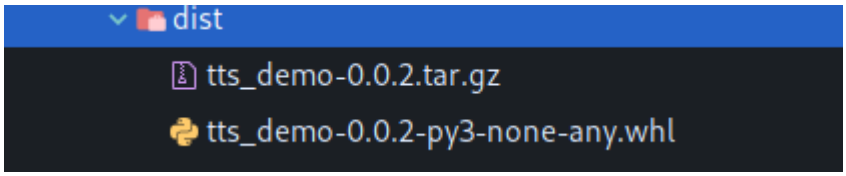
```
        setup_py_pkg "your py project directory"
```

Of course, you can also use the AlphaMini inner tool kit.

- Mode 3: python code packaging

```
        import mini.pkg_tool as Tool
        # Package the tts_demo project in the current directory into a py wheel, the return
value is the path of whl after packaging.
        pkg_path = Tool.setup_py_pkg("tts_demo")
        print(f'{pkg_path}')
```

- Take tts_demo as an example, after successful packaging, a dist directory is created, *.whl file is the file that can be installed, as shown below:



3.2 Installation

To install a packaged `*.whl` program into a robot, make sure the robot and the PC are on the same LAN. Upload the `.whl` file and install it inside the robot (make the robot execute `pip install xx.whl` command first), fill in the last 4 digits of the robot serial number;

- Mode 1: Install the robot via AlphaMini command line, `--type` specify the robot type. The AlphaMini sdk searches for the china educational version robot by default. so oversea users should reset robot type to `edu`.

  `edu` - for AlphaMini overseas education version.

```
install_py_pkg "packaged .whl path" "robot serial number" --type "edu"
```

- Mode 2: via python code

```
import mini.pkg_tool as Tool
from mini import mini_sdk as MiniSdk
# Set the robot type
MiniSdk.set_robot_type(MiniSdk.RobotType.EDU)
# Install the file simple_socket-0.0.2-py3-none-any.whl in the current directory.
Tool.install_py_pkg(package_path="simple_socket-0.0.2-py3-none-any.whl",
robot_id="robot_number")
```

3.3 Query

Query the information of the `py` program installed in the robot, send the command `pip list` to the robot and return the result of the command.

3.3.1 List `py` modules

List in-robot modules

- Mode 1: List `py` modules by command line. The AlphaMini sdk searches for the china educational version robot by default. so oversea users should specify robot type to `edu`.

  `edu` - for AlphaMini overseas education version.

```
list_py_pkg "0090" --type "edu"
```

- Mode 2: By python code

```
import mini.pkg_tool as Tool
from mini import mini_sdk as MiniSdk
# Set the robot type
MiniSdk.set_robot_type(MiniSdk.RobotType.EDU)
# List the py packages in the robot
list_info = Tool.list_py_pkg(robot_id="0090")
print(f'{list_info}')
```

3.3.2 Query the specification of the py module

Running command `pip show xx` to query specification of the py module. For example as blow, xx stands for `tts_demo`. Make sure the robot and the PC are on the same LAN, specify the last 4 digits of robot serial number.

- Mode 1: By command line

```
query_py_pkg "tts_demo" "0090" --type "edu"
```

- Mode 2: By python code

```
import mini.pkg_tool as Tool
from mini import mini_sdk as MiniSdk
# Set the robot type
MiniSdk.set_robot_type(MiniSdk.RobotType.EDU)
# Query tts_demo package information
info = Tool.query_py_pkg(pkg_name="tts_demo", robot_id="0090")
print(f'{info}')
```

3.4 Unload

Running command `pip uninstall -y xx` to uninstall the program installed on the robot. For example as blow, xx stands for `tts_demo`; make sure that the robot and the PC are on the same network, input the last 4 digits of robot serial number.

- Mode 1: by command line

```
uninstall_py_pkg "tts_demo" "0090" --type "edu"
```

- Mode 2: via python code

```
import mini.pkg_tool as Tool
from mini import mini_sdk as MiniSdk
# Set the robot type
MiniSdk.set_robot_type(MiniSdk.RobotType.EDU)
# Uninstall tts_demo
Tool.uninstall_py_pkg(pkg_name="tts_demo", robot_id="0090")
```

3.5 Operation

`py` program can be run in robot, send `xx` command to robot and get the result. In the following example, xx stands for `tts_demo`; make sure the robot and the PC are on the same LAN, because you need to get the robot to execute the command. Fill in the last 4 digits of the robot's serial number.

- Mode 1: by command line.

```
# Triggers the tts_demo program to run.
run_py_pkg "tts_demo" "0090" --type "edu"
```

- Mode 2: via python code

```
import mini.pkg_tool as Tool
from mini import mini_sdk as MiniSdk
# Set the robot type
MiniSdk.set_robot_type(MiniSdk.RobotType.EDU)
# Triggers tts_demo offline execution
Tool.run_py_pkg("tts_demo", robot_id="0090")
```

3.6 Executing shell commands

From the description above, you may know that the robot has a built-in terminal environment (termux), python is one of executive programs, send `pip install/uninstall alphamini` command to the robot, install/uninstall third-party python libraries.

- Mode 1: via command line, install/uninstall AlphaMini sdk.

```
run_cmd "pip install alphamini" "0090" --type "edu"
run_cmd "pip uninstall -y alphamini" "0090" --type "edu"
```

- Mode 2: via python code, install/uninstall AlphaMini python sdk

```python
import mini.pkg_tool as Tool
from mini import mini_sdk as MiniSdk
# Set the robot type
MiniSdk.set_robot_type(MiniSdk.RobotType.EDU)
# Execute a shell command
Tool.run_py_pkg("pip install alphamini", robot_id="0090")
Tool.run_py_pkg("pip uninstall -y alpahmini", robot_id="0090")
```

You can also send the command `pkg install -f xx` to the robot to install other third-party executable programs or libraries, in the following example `xx` is the name of the third-party library, and `pkg` is a package extension command built into the robot.

```
run_cmd "pkg install -f xx" "0090" --type "edu"
run_cmd "pkg uninstall -f xx" "0090" --type "edu"
```